DATA STRUCTURES AND ALGORITHMS

23CSE203

LAB MANUAL

SUBMITTED BY	
NAME	KORRAPATI KEERTHAN KARTHIKEYA
ROLL NO	AV.SC.U4CSE25211
YEAR/SEM/SECTION	2ND YEAR/SEM-3/CSE-C
CREDITS	05

SUBMITTED TO	Dr Gayathri Parasa
DESIGNATION	Asst. Professor (Sr. Gr)
DEPARTMENT	SCHOOL OF COMPUTING

INDEX

S.NO	TITLE	PAGE NO
1.	**WEEK 1:** **1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!". **1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n(power) where: P= principal growth factor, n= number of years. **1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **1(E).** Write a recursive python program to print the first n terms of the Fibonacci series.	5–14
2.	**WEEK 2:** **2(A).** Write a python program to implement linear search. **2(B).** Write a python program to implement binary search with and without sorted array as an input.	15–18
3.	**WEEK-3:** **3(A).** Write a python program to implement bubble sort. **3(B).** Write a python program to implement insertion sort. **3(C).** Write a python program to implement selection sort.	19–24
4.	**WEEK-4:** **4(A).** Write a python program to implement quick sort. **4(B).** Write a python program to implement merge sort.	25–28
5.	**WEEK-5:** **5(A).** Write a python program to implement a Singly Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index	29–32

	5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Sort the data of SLL 10) Search data in SLL 11) Display or Traverse 12) Exit	
6.	**WEEK-6:** Write a python program to implement a Double Linked List with the below operations. 1) Create Linked List 2) Insert Beginning 3) Insert End 4) Insert at Index 5) Delete by Value 6) Delete First Node 7) Delete Last Node 8) Count number of nodes 9) Display or Traverse 10) Exit	33–36
7.	**WEEK-7:** **7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array. **7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list.	37–40
8.	**WEEK-8:** **8(A).** Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. **8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.	41–46
9.	**WEEK-9:** Write a python program to implement a Binary Search Tree (BST) and perform the following operations. 1. Insertion 2. Deletion	47–50

	3. Tree Traversals – in-order, pre-order and post-order 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	
10.	**WEEK-10:** Write a menu driven python program to implement an AVL Tree and perform the following operations. 1. Insertion 2. Deletion 3. Tree Traversals 4. Count Leaf Nodes 5. Count Internal Nodes 6. Height of the Tree	51–54
11.	**WEEK-11:** Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations: 1. Insertion of an element into the Heap. 2. Deletion of the root element from the Heap. 3. Display the array representation of the Heap. 4. Exit the program.	55–57
12.	**WEEK-12:** Write a program to implement the following three collision resolution techniques in a Hash Table: 1. Separate Chaining 2. Linear Probing 3. Quadratic Probing	58–60

WEEK – 1

PROGRAM-1(A):

Recursive Countdown

QUESTION:

1(A). A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by “Launch!”.

AIM:

To use a recursive function to print a countdown from a given number and then display Launch!.

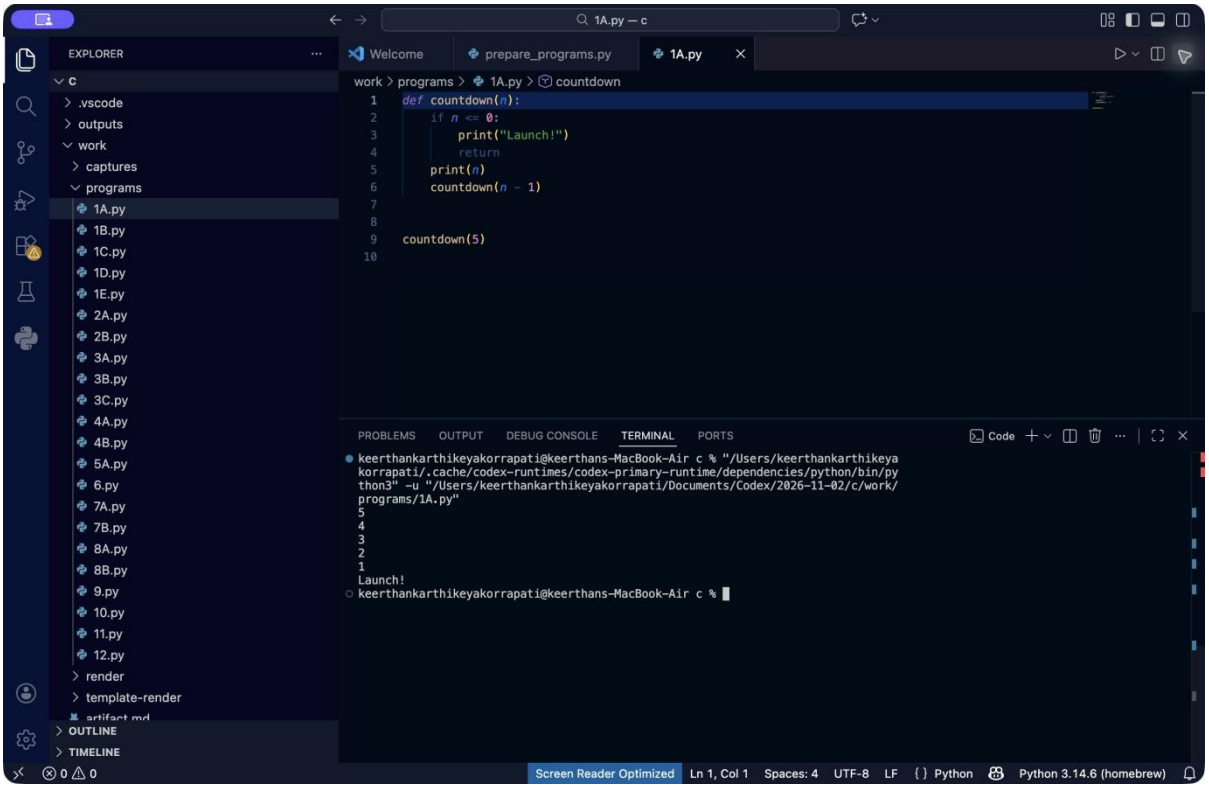
CODE:

```
def countdown(n):
    if n <= 0:
        print("Launch!")
        return
    print(n)
    countdown(n - 1)


countdown(5)
```

OUTPUT:

VS Code run for Program 1(A)



PROGRAM-1(B):

Recursive Power

QUESTION:

1(B). Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate Pn(power) where: P= principal growth factor, n= number of years.

AIM:

To calculate a non-negative integer power recursively using the relation P^n = P × P^(n-1).

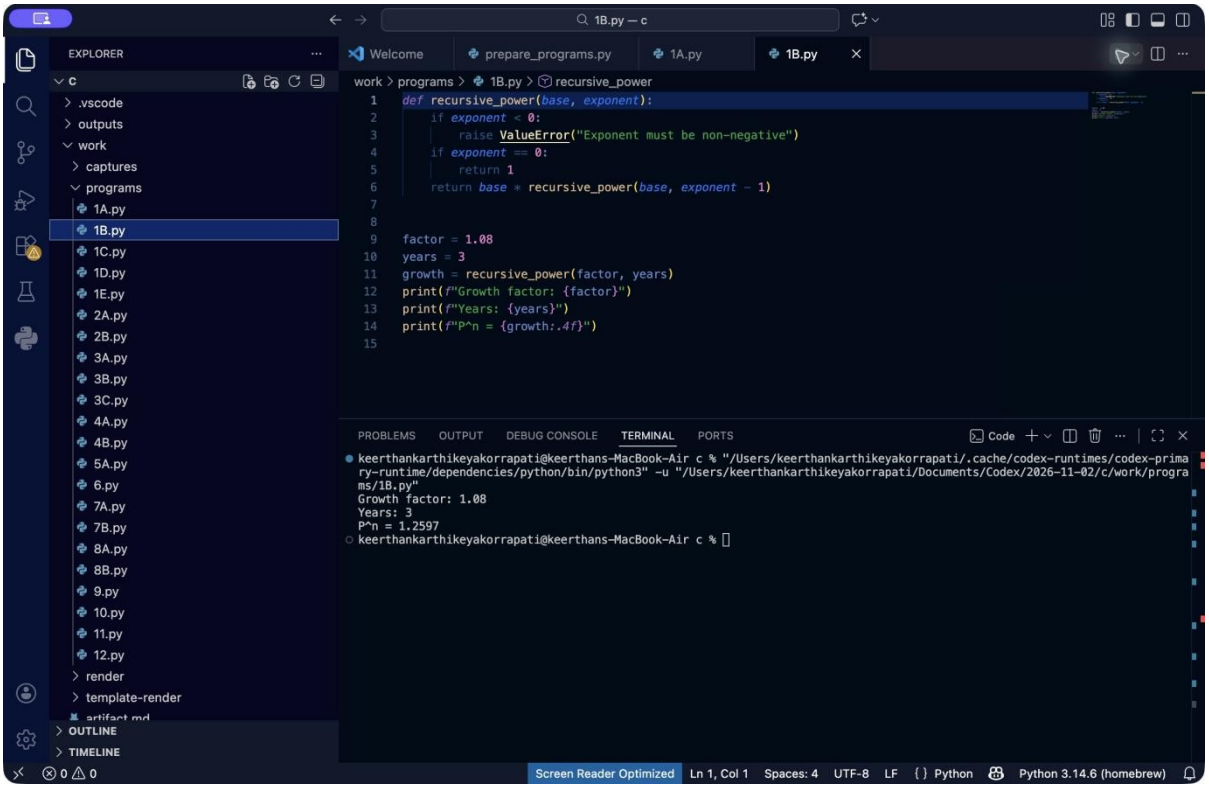
CODE:

```
def recursive_power(base, exponent):
    if exponent < 0:
        raise ValueError("Exponent must be non-negative")
    if exponent == 0:
        return 1
    return base * recursive_power(base, exponent - 1)


factor = 1.08
years = 3
growth = recursive_power(factor, years)
print(f"Growth factor: {factor}")
print(f"Years: {years}")
print(f"P^n = {growth:.4f}")
```

OUTPUT:

VS Code run for Program 1(B)



PROGRAM-1(C):

Recursive Employee ID Search

QUESTION:

1(C). A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

AIM:

To search for an employee ID recursively and return its index, or -1 when it is absent.

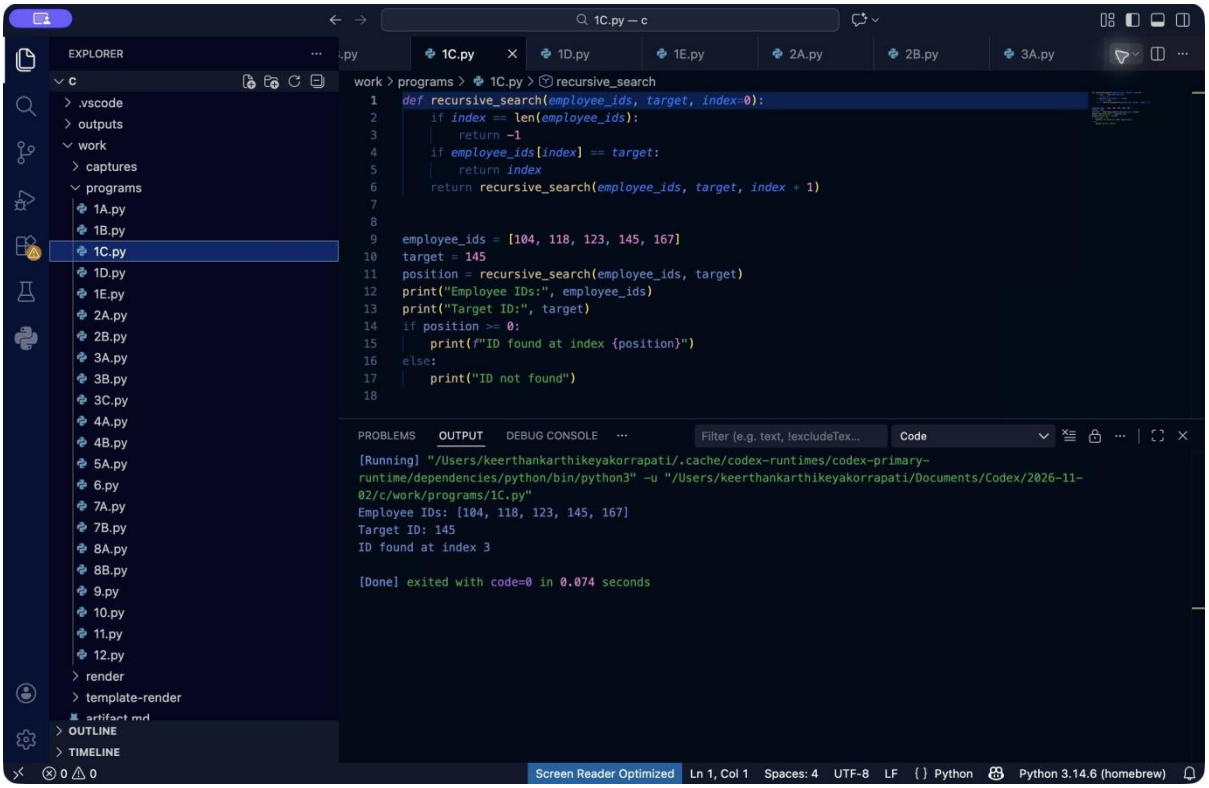
CODE:

```
def recursive_search(employee_ids, target, index=0):
    if index == len(employee_ids):
        return –1
    if employee_ids[index] == target:
        return index
    return recursive_search(employee_ids, target, index + 1)


employee_ids = [104, 118, 123, 145, 167]
target = 145
position = recursive_search(employee_ids, target)
print("Employee IDs:", employee_ids)
print("Target ID:", target)
if position >= 0:
    print(f"ID found at index {position}")
else:
    print("ID not found")
```

OUTPUT:

VS Code run for Program 1(C)



PROGRAM-1(D):

Recursive Factorial

QUESTION:

1(D). In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

AIM:

To compute n! recursively, using 0! = 1 as the base case.

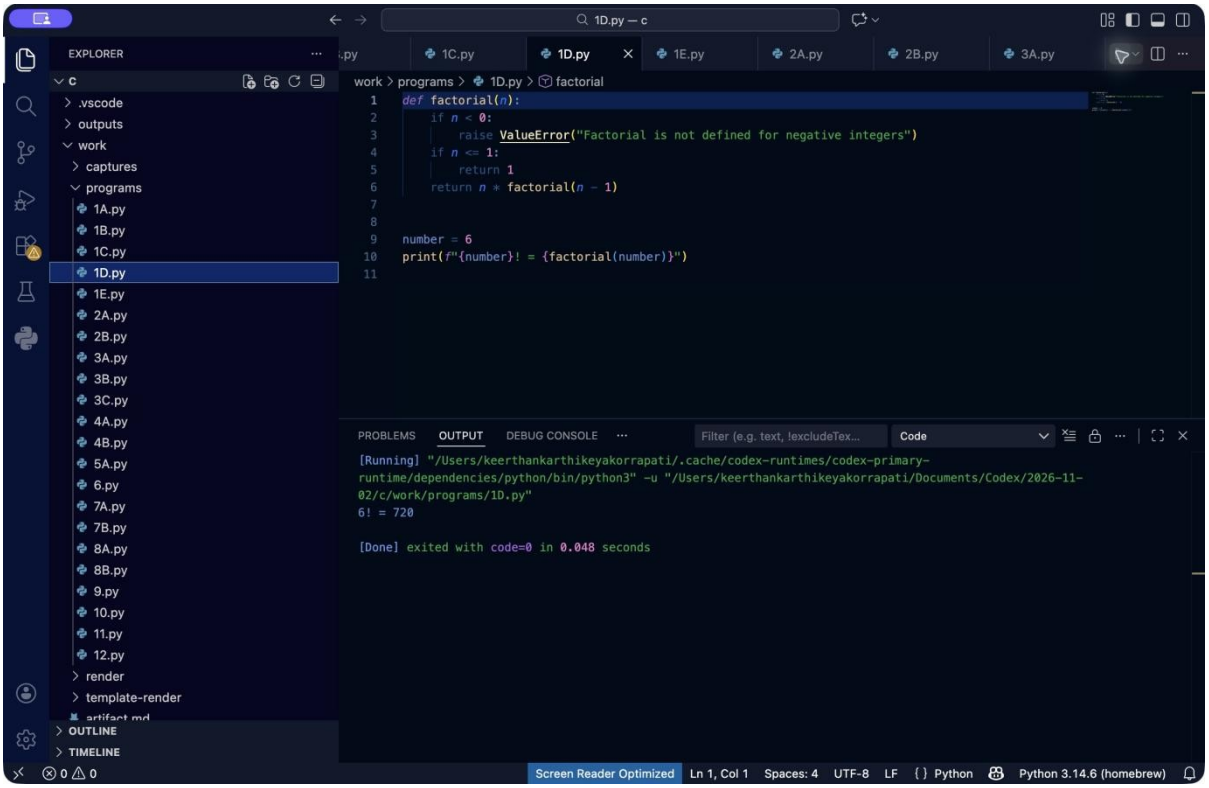
CODE:

```
def factorial(n):
    if n < 0:
        raise ValueError("Factorial is not defined for negative integers")
    if n <= 1:
        return 1
    return n * factorial(n - 1)


number = 6
print(f"{number}! = {factorial(number)}")
```

OUTPUT:

VS Code run for Program 1(D)



PROGRAM-1(E):

Recursive Fibonacci Series

QUESTION:

1(E). Write a recursive python program to print the first n terms of the Fibonacci series.

AIM:

To generate the first n Fibonacci numbers recursively, beginning with 0 and 1.

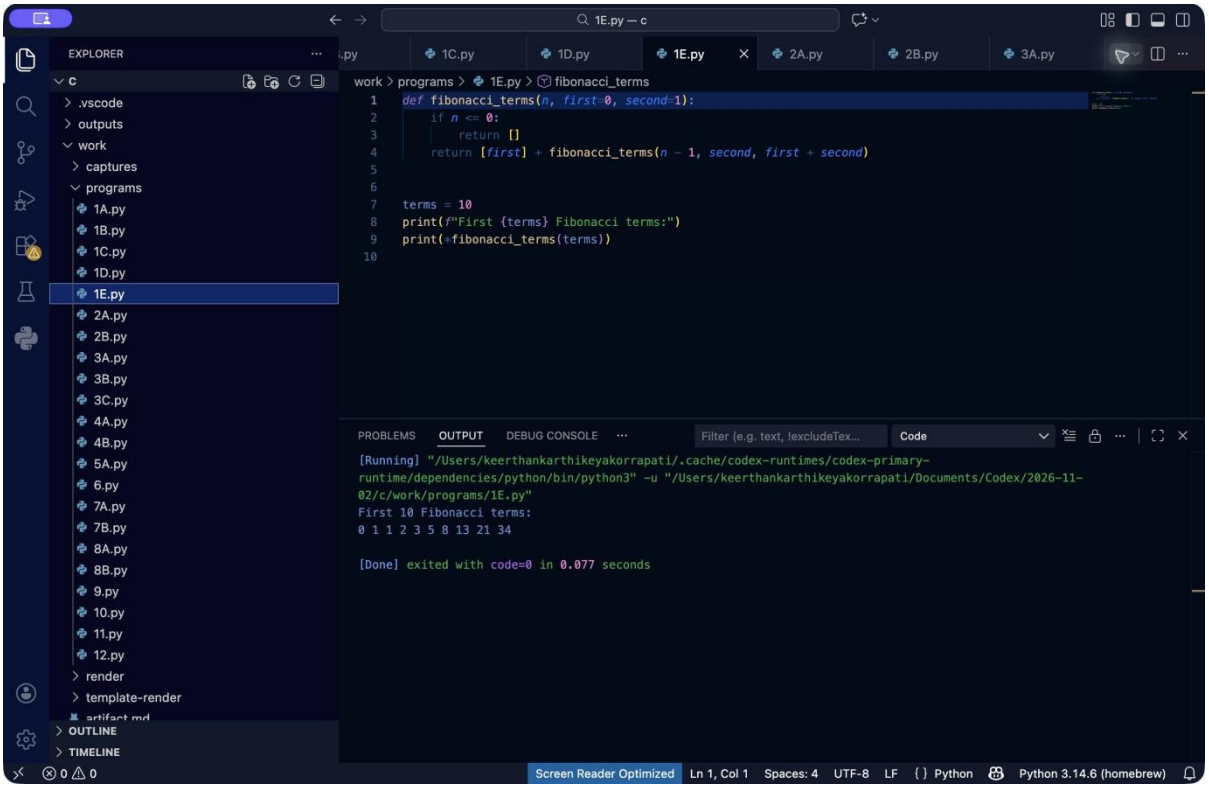
CODE:

```
def fibonacci_terms(n, first=0, second=1):
    if n <= 0:
        return []
    return [first] + fibonacci_terms(n - 1, second, first + second)


terms = 10
print(f"First {terms} Fibonacci terms:")
print(*fibonacci_terms(terms))
```

OUTPUT:

VS Code run for Program 1(E)



WEEK – 2

PROGRAM-2(A):

Linear Search

QUESTION:

2(A). Write a python program to implement linear search.

AIM:

To check each element of a list in order and report the position of a target value.

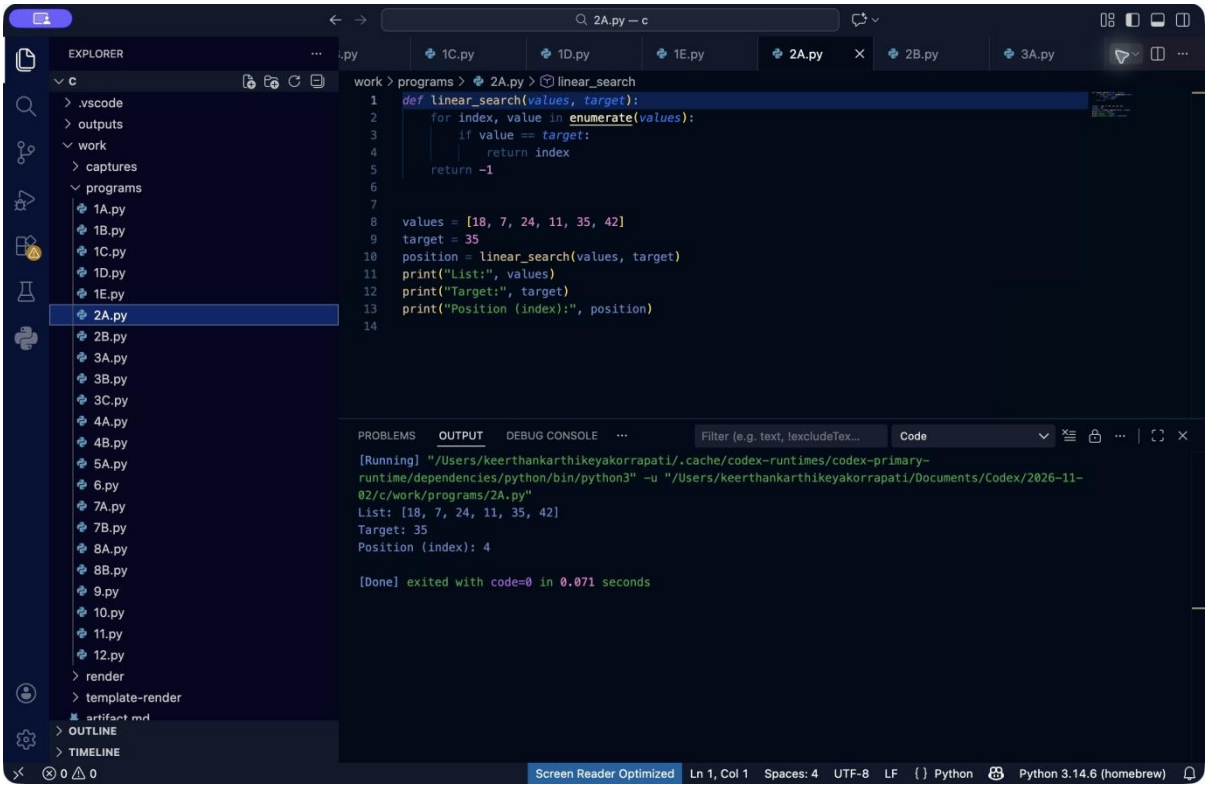
CODE:

```
def linear_search(values, target):
    for index, value in enumerate(values):
        if value == target:
            return index
    return -1


values = [18, 7, 24, 11, 35, 42]
target = 35
position = linear_search(values, target)
print("List:", values)
print("Target:", target)
print("Position (index):", position)
```

OUTPUT:

VS Code run for Program 2(A)



PROGRAM-2(B):

Binary Search on Sorted and Unsorted Input

QUESTION:

2(B). Write a python program to implement binary search with and without sorted array as an input.

AIM:

To perform binary search on an already sorted list or sort an unsorted list before searching.

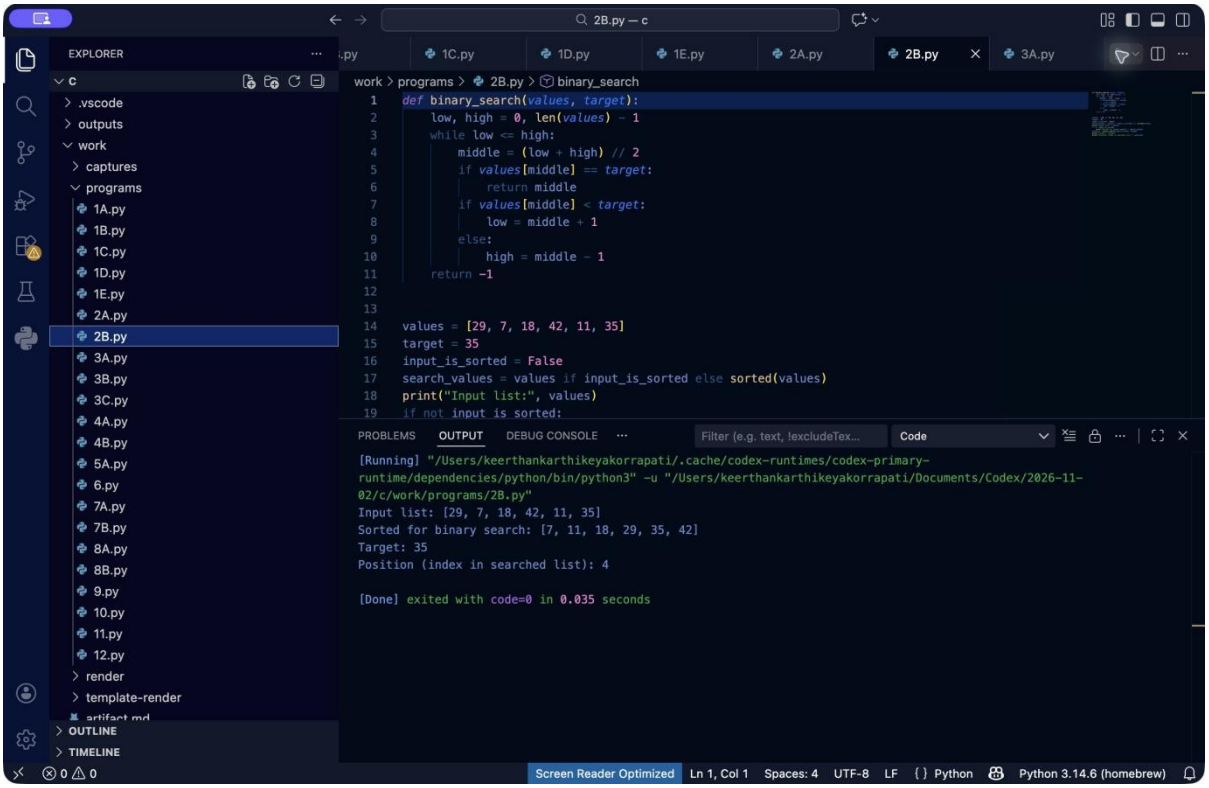
CODE:

```
def binary_search(values, target):
    low, high = 0, len(values) - 1
    while low <= high:
        middle = (low + high) // 2
        if values[middle] == target:
            return middle
        if values[middle] < target:
            low = middle + 1
        else:
            high = middle - 1
    return –1


values = [29, 7, 18, 42, 11, 35]
target = 35
input_is_sorted = False
search_values = values if input_is_sorted else sorted(values)
print("Input list:", values)
if not input_is_sorted:
    print("Sorted for binary search:", search_values)
position = binary_search(search_values, target)
print("Target:", target)
print("Position (index in searched list):", position)
```

OUTPUT:

VS Code run for Program 2(B)



WEEK – 3

PROGRAM-3(A):

Bubble Sort

QUESTION:

3(A). Write a python program to implement bubble sort.

AIM:

To sort a list in ascending order by repeatedly swapping adjacent out-of-order elements.

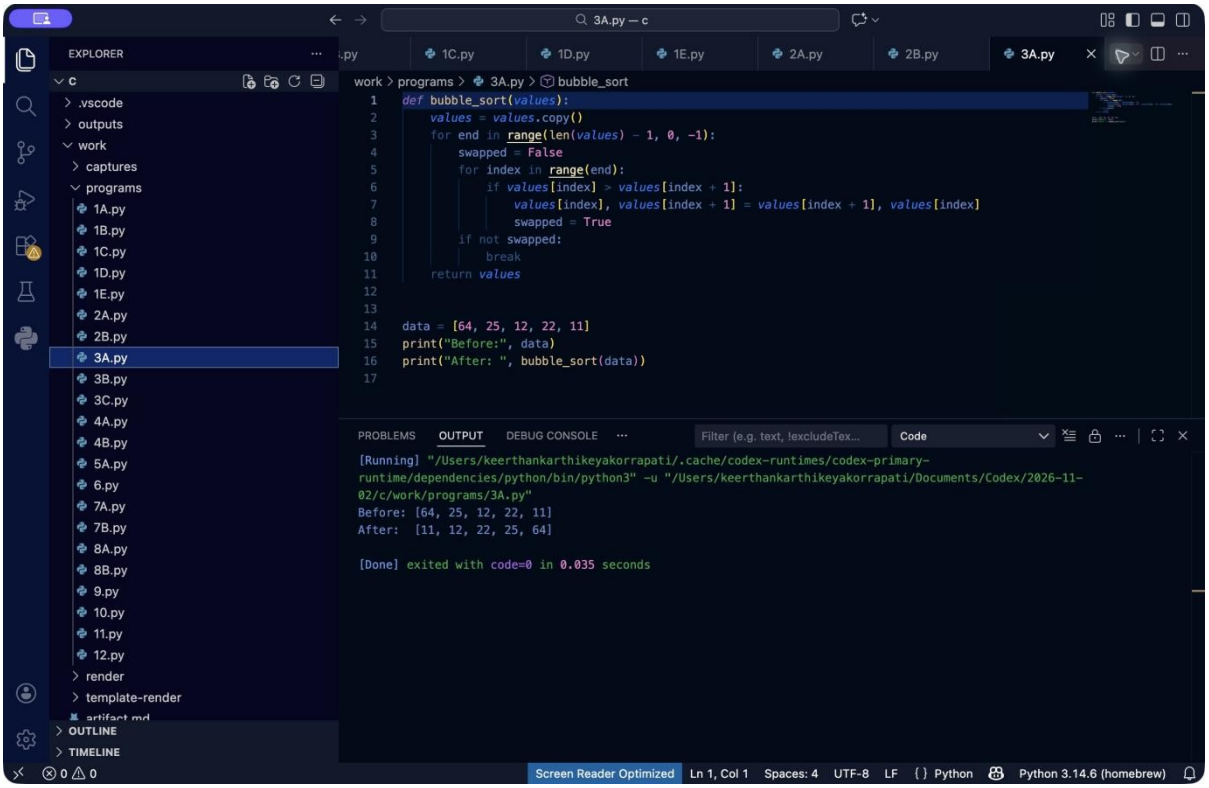
CODE:

```
def bubble_sort(values):
    values = values.copy()
    for end in range(len(values) - 1, 0, −1):
        swapped = False
        for index in range(end):
            if values[index] > values[index + 1]:
                values[index], values[index + 1] = values[index + 1], values[index]
                swapped = True
        if not swapped:
            break
    return values


data = [64, 25, 12, 22, 11]
print("Before:", data)
print("After: ", bubble_sort(data))
```

OUTPUT:

VS Code run for Program 3(A)



PROGRAM-3(B):

Insertion Sort

QUESTION:

3(B). Write a python program to implement insertion sort.

AIM:

To build a sorted list one element at a time by inserting each value into its correct position.

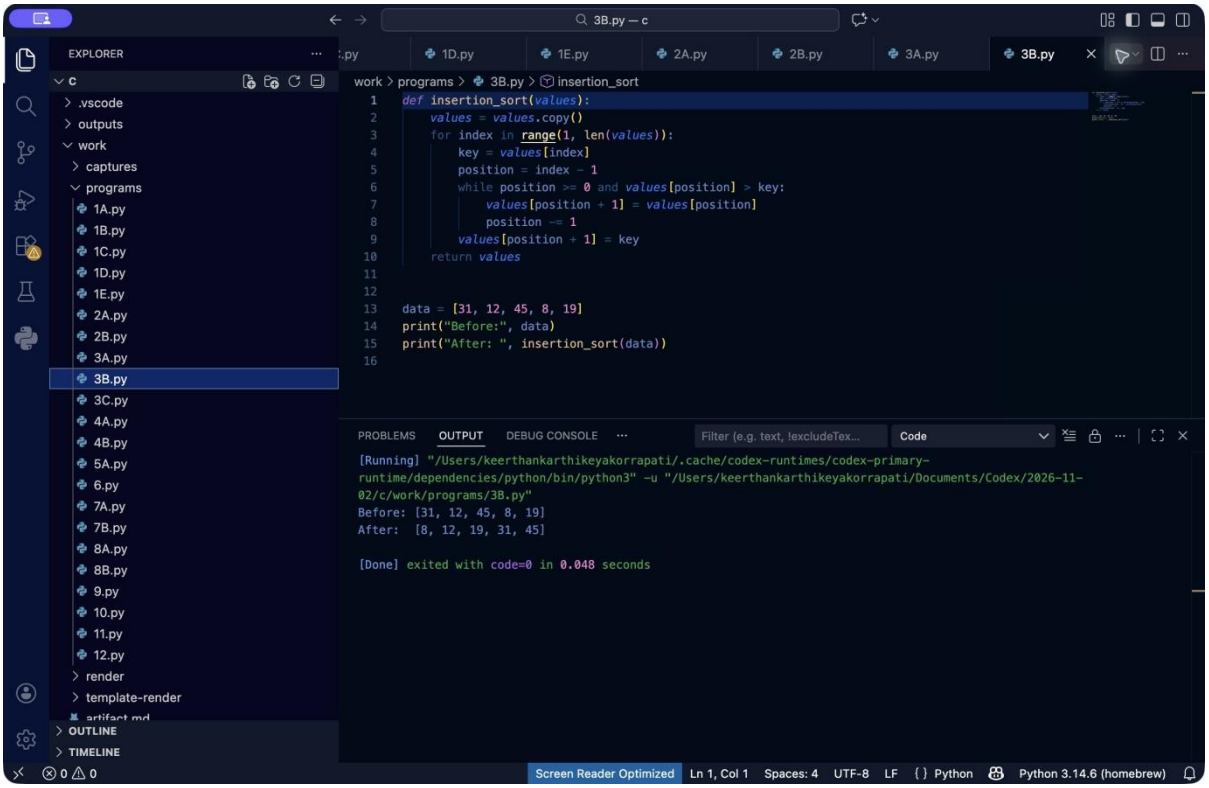
CODE:

```
def insertion_sort(values):
    values = values.copy()
    for index in range(1, len(values)):
        key = values[index]
        position = index - 1
        while position >= 0 and values[position] > key:
            values[position + 1] = values[position]
            position -= 1
        values[position + 1] = key
    return values


data = [31, 12, 45, 8, 19]
print("Before:", data)
print("After: ", insertion_sort(data))
```

OUTPUT:

VS Code run for Program 3(B)



PROGRAM-3(C):

Selection Sort

QUESTION:

3(C). Write a python program to implement selection sort.

AIM:

To sort a list by repeatedly selecting the smallest remaining element and placing it next.

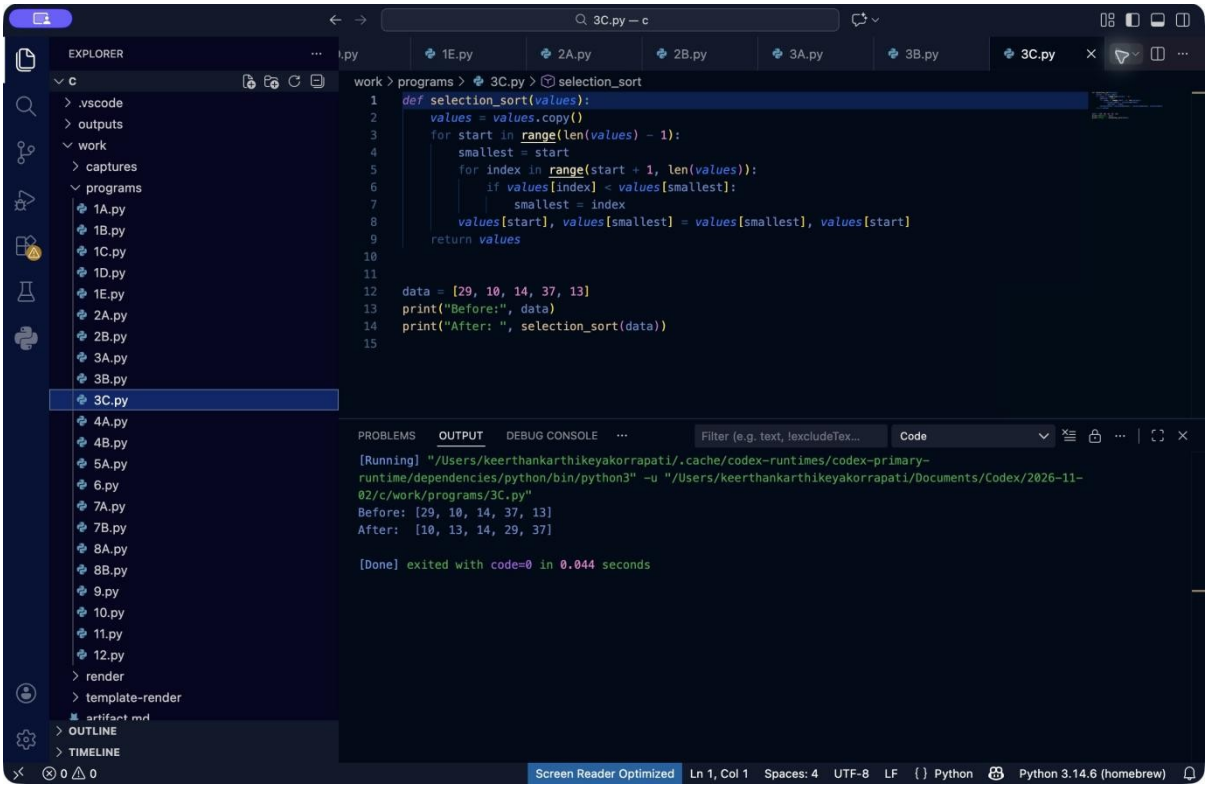
CODE:

```
def selection_sort(values):
    values = values.copy()
    for start in range(len(values) - 1):
        smallest = start
        for index in range(start + 1, len(values)):
            if values[index] < values[smallest]:
                smallest = index
        values[start], values[smallest] = values[smallest], values[start]
    return values


data = [29, 10, 14, 37, 13]
print("Before:", data)
print("After: ", selection_sort(data))
```

OUTPUT:

VS Code run for Program 3(C)



WEEK – 4

PROGRAM-4(A):

Quick Sort

QUESTION:

4(A). Write a python program to implement quick sort.

AIM:

To sort a list recursively by partitioning values around a pivot.

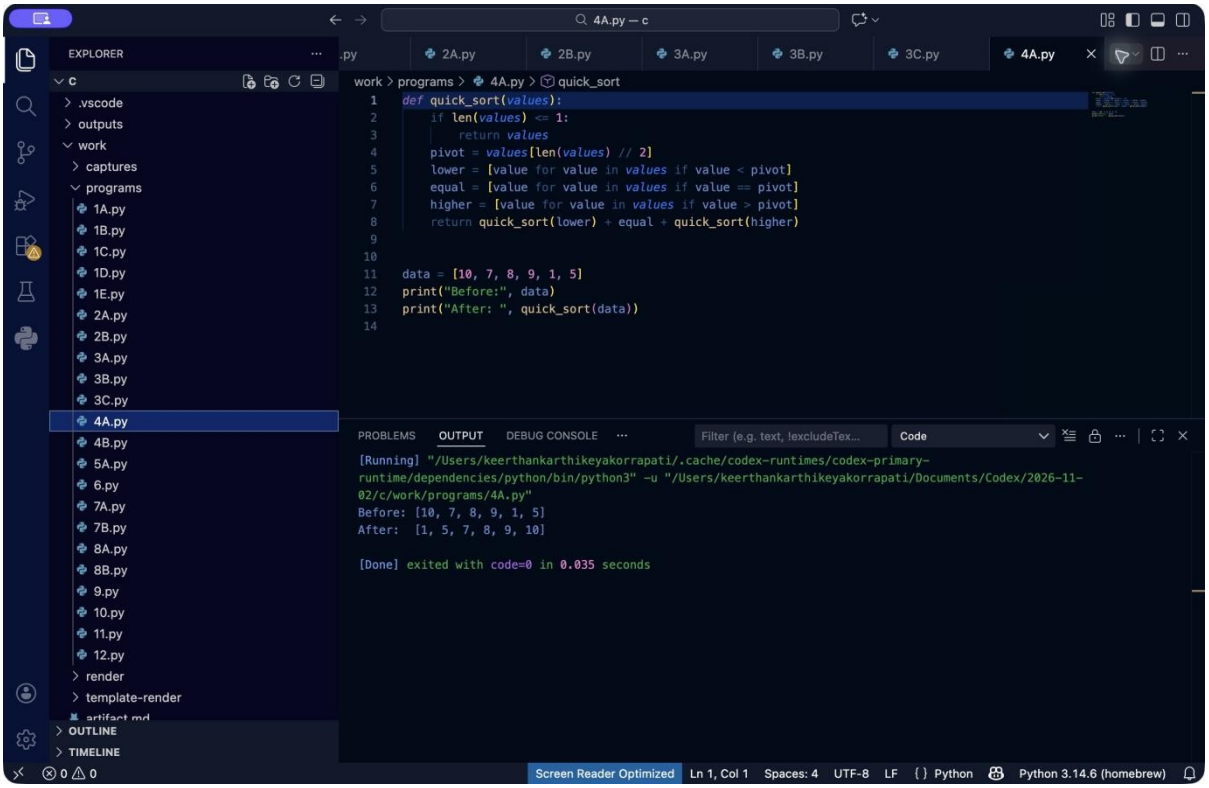
CODE:

```
def quick_sort(values):
    if len(values) <= 1:
        return values
    pivot = values[len(values) // 2]
    lower = [value for value in values if value < pivot]
    equal = [value for value in values if value == pivot]
    higher = [value for value in values if value > pivot]
    return quick_sort(lower) + equal + quick_sort(higher)


data = [10, 7, 8, 9, 1, 5]
print("Before:", data)
print("After: ", quick_sort(data))
```

OUTPUT:

VS Code run for Program 4(A)



PROGRAM-4(B):

Merge Sort

QUESTION:

4(B). Write a python program to implement merge sort.

AIM:

To sort a list recursively by splitting it into halves and merging the sorted halves.

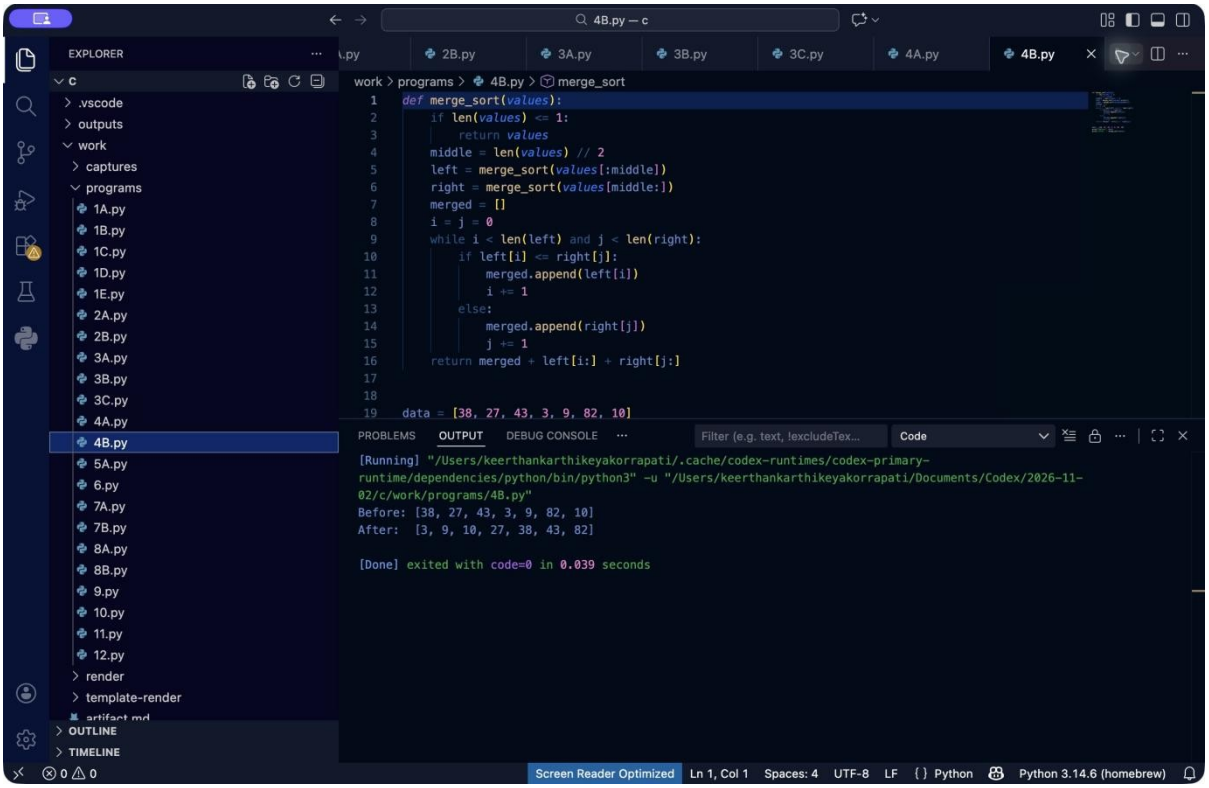
CODE:

```
def merge_sort(values):
    if len(values) <= 1:
        return values
    middle = len(values) // 2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    return merged + left[i:] + right[j:]


data = [38, 27, 43, 3, 9, 82, 10]
print("Before:", data)
print("After: ", merge_sort(data))
```

OUTPUT:

VS Code run for Program 4(B)



WEEK – 5

PROGRAM-5(A):

Singly Linked List Operations

QUESTION:

5(A). Write a python program to implement a Singly Linked List with the below operations.
Create Linked List
Insert Beginning
Insert End
Insert at Index
Delete by Value
Delete First Node
Delete Last Node
Count number of nodes
Sort the data of SLL
Search data in SLL
Display or Traverse
Exit

AIM:

To implement a menu-driven singly linked list and perform the required insertion, deletion, search, sorting, counting, and traversal operations.

CODE:

```
class Node:
    def __init__(self, value):
        self.value, self.next = value, None


class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create(self, values):
        self.head = None
        for value in values:
            self.insert_end(value)

    def insert_beginning(self, value):
        node = Node(value)
        node.next, self.head = self.head, node

    def insert_end(self, value):
        node = Node(value)
        if self.head is None:
            self.head = node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = node

    def insert_at(self, index, value):
```

```
        if index < 0:
            raise IndexError("Index cannot be negative")
        if index == 0:
            self.insert_beginning(value)
            return
        current = self.head
        for _ in range(index - 1):
            if current is None:
                raise IndexError("Index is out of range")
            current = current.next
        if current is None:
            raise IndexError("Index is out of range")
        node = Node(value)
        node.next, current.next = current.next, node

    def delete_value(self, value):
        current, previous = self.head, None
        while current and current.value != value:
            previous, current = current, current.next
        if current is None:
            return False
        if previous is None:
            self.head = current.next
        else:
            previous.next = current.next
        return True

    def delete_first(self):
        if self.head is None:
            return None
        value, self.head = self.head.value, self.head.next
        return value

    def delete_last(self):
        if self.head is None:
            return None
        if self.head.next is None:
            return self.delete_first()
        current = self.head
        while current.next.next:
            current = current.next
        value, current.next = current.next.value, None
        return value

    def values(self):
        result, current = [], self.head
        while current:
            result.append(current.value)
            current = current.next
        return result

    def count(self):
        return len(self.values())

    def sort(self):
        self.create(sorted(self.values()))

    def search(self, target):
        for index, value in enumerate(self.values()):
            if value == target:
                return index
        return -1

    def display(self):
        print(" -> ".join(map(str, self.values())) or "List is empty")

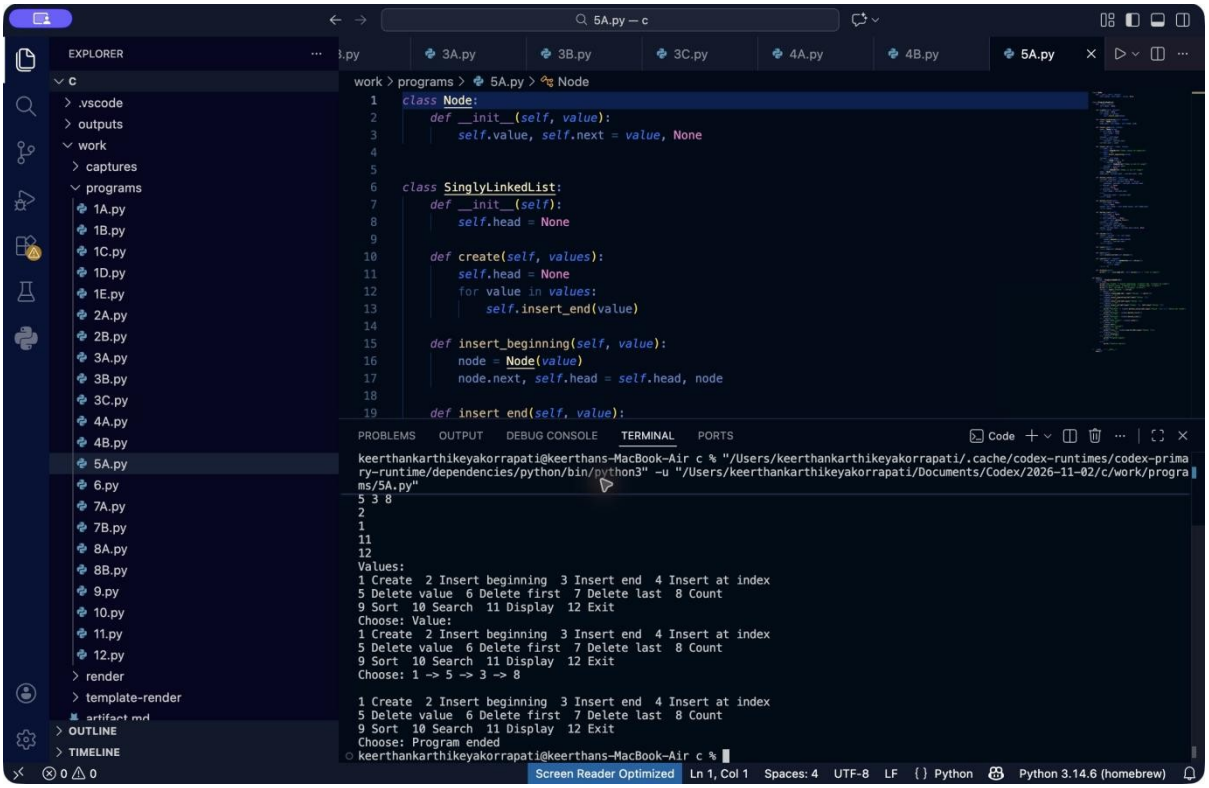

def main():
    linked = SinglyLinkedList()
```

```
    while True:
        print("\n1 Create  2 Insert beginning  3 Insert end  4 Insert at index")
        print("5 Delete value  6 Delete first  7 Delete last  8 Count")
        print("9 Sort  10 Search  11 Display  12 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            linked.create(map(int, input("Values: ").split()))
        elif choice == "2":
            linked.insert_beginning(int(input("Value: ")))
        elif choice == "3":
            linked.insert_end(int(input("Value: ")))
        elif choice == "4":
            linked.insert_at(int(input("Index: ")), int(input("Value: ")))
        elif choice == "5":
            print("Deleted" if linked.delete_value(int(input("Value: "))) else "Value not found")
        elif choice == "6":
            print("Deleted:", linked.delete_first())
        elif choice == "7":
            print("Deleted:", linked.delete_last())
        elif choice == "8":
            print("Node count:", linked.count())
        elif choice == "9":
            linked.sort()
            print("List sorted")
        elif choice == "10":
            print("Index:", linked.search(int(input("Value: "))))
        elif choice == "11":
            linked.display()
        elif choice == "12":
            print("Program ended")
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 5(A)



WEEK – 6

PROGRAM-6:

Doubly Linked List Operations

QUESTION:

Write a Python program to implement a Doubly Linked List with these operations: create; insert at the beginning, end, or index; delete by value, first node, or last node; count nodes; display/traverse; and exit.

AIM:

To build a menu-driven doubly linked list whose nodes link to both the previous and next nodes.

CODE:

```
class Node:
    def __init__(self, value):
        self.value, self.previous, self.next = value, None, None


class DoublyLinkedList:
    def __init__(self):
        self.head = self.tail = None

    def create(self, values):
        self.head = self.tail = None
        for value in values:
            self.insert_end(value)

    def insert_beginning(self, value):
        node = Node(value)
        node.next = self.head
        if self.head:
            self.head.previous = node
        else:
            self.tail = node
        self.head = node

    def insert_end(self, value):
        node = Node(value)
        node.previous = self.tail
        if self.tail:
            self.tail.next = node
        else:
            self.head = node
        self.tail = node

    def insert_at(self, index, value):
        if index < 0:
            raise IndexError("Index cannot be negative")
        if index == 0:
            self.insert_beginning(value)
            return
        current = self.head
        for _ in range(index):
            if current is None:
                raise IndexError("Index is out of range")
            current = current.next
        if current is None:
            self.insert_end(value)
            return
```

```
        node = Node(value)
        node.previous, node.next = current.previous, current
        current.previous.next = node
        current.previous = node

    def _unlink(self, node):
        if node.previous:
            node.previous.next = node.next
        else:
            self.head = node.next
        if node.next:
            node.next.previous = node.previous
        else:
            self.tail = node.previous
        return node.value

    def delete_value(self, value):
        current = self.head
        while current and current.value != value:
            current = current.next
        return self._unlink(current) if current else None

    def delete_first(self):
        return self._unlink(self.head) if self.head else None

    def delete_last(self):
        return self._unlink(self.tail) if self.tail else None

    def count(self):
        total, current = 0, self.head
        while current:
            total += 1
            current = current.next
        return total

    def display(self):
        values, current = [], self.head
        while current:
            values.append(str(current.value))
            current = current.next
        print(" <-> ".join(values) or "List is empty")

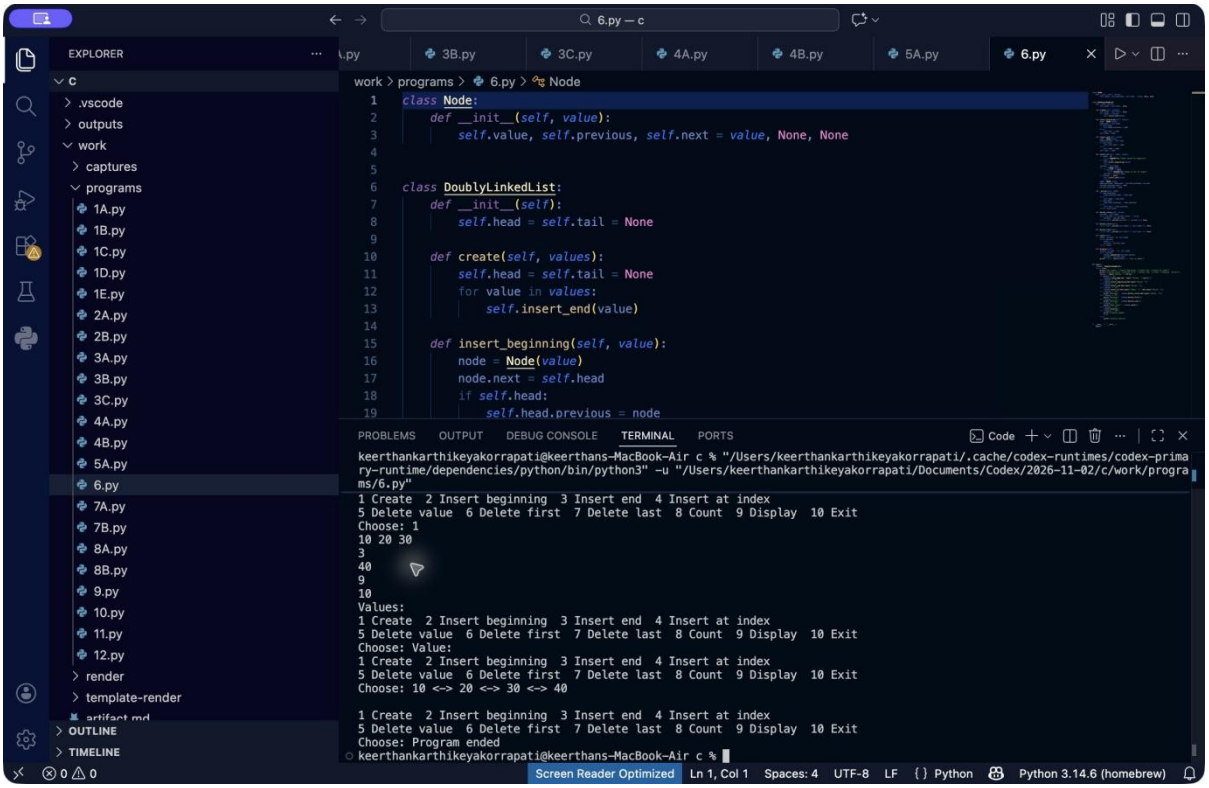

def main():
    linked = DoublyLinkedList()
    while True:
        print("\n1 Create  2 Insert beginning  3 Insert end  4 Insert at index")
        print("5 Delete value  6 Delete first  7 Delete last  8 Count  9 Display  10 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            linked.create(map(int, input("Values: ").split()))
        elif choice == "2":
            linked.insert_beginning(int(input("Value: ")))
        elif choice == "3":
            linked.insert_end(int(input("Value: ")))
        elif choice == "4":
            linked.insert_at(int(input("Index: ")), int(input("Value: ")))
        elif choice == "5":
            print("Deleted:", linked.delete_value(int(input("Value: "))))
        elif choice == "6":
            print("Deleted:", linked.delete_first())
        elif choice == "7":
            print("Deleted:", linked.delete_last())
        elif choice == "8":
            print("Node count:", linked.count())
        elif choice == "9":
            linked.display()
        elif choice == "10":
            print("Program ended")
```

```
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 6



WEEK – 7

PROGRAM-7(A):

Stack Using an Array

QUESTION:

7(A). Write a python program to implement stack operations such as push, pop, peek, display using an array.

AIM:

To implement a last-in, first-out stack using a Python list and a menu for push, pop, peek, and display.

CODE:

```
class ArrayStack:
    def __init__(self):
        self.items = []

    def push(self, value):
        self.items.append(value)

    def pop(self):
        return self.items.pop() if self.items else None

    def peek(self):
        return self.items[-1] if self.items else None

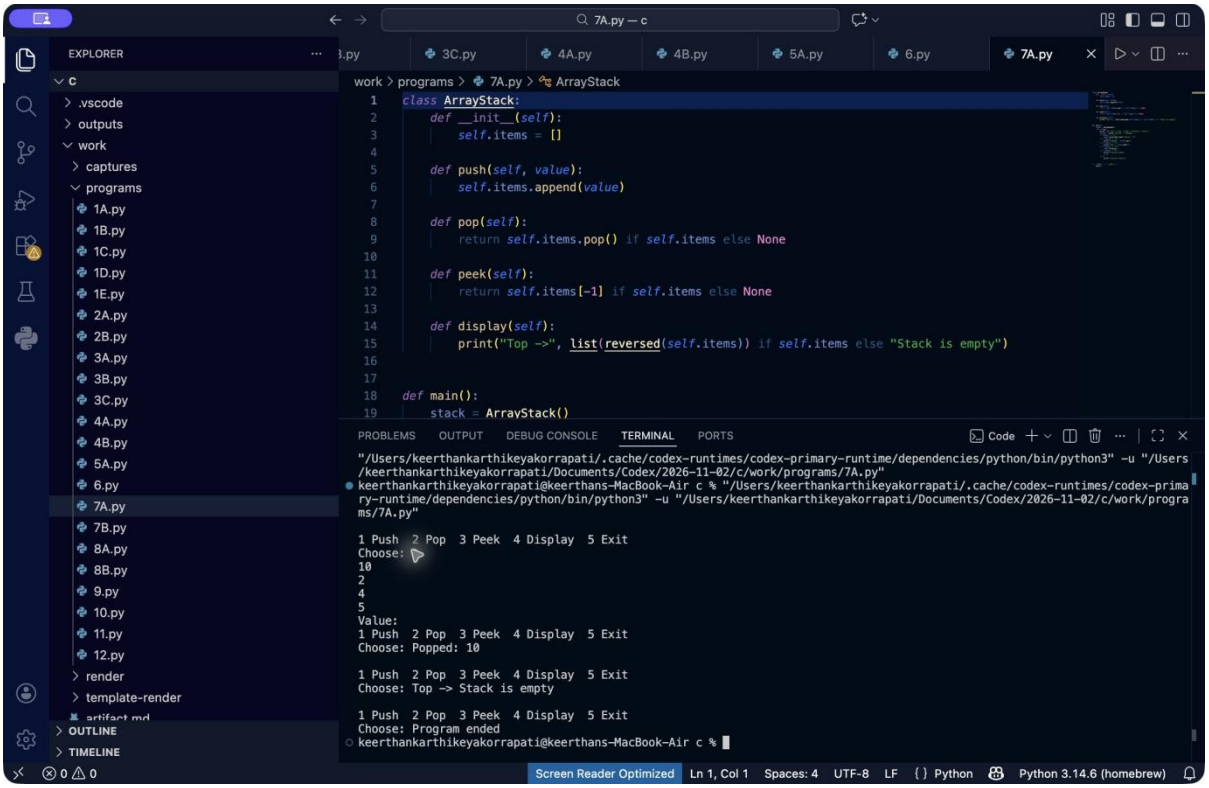
    def display(self):
        print("Top ->", list(reversed(self.items)) if self.items else "Stack is empty")


def main():
    stack = ArrayStack()
    while True:
        print("\n1 Push  2 Pop  3 Peek  4 Display  5 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            stack.push(int(input("Value: ")))
        elif choice == "2":
            print("Popped:", stack.pop())
        elif choice == "3":
            print("Top:", stack.peek())
        elif choice == "4":
            stack.display()
        elif choice == "5":
            print("Program ended")
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 7(A)



PROGRAM-7(B):

Stack Using a Linked List

QUESTION:

7(B). Write a python program to implement stack operations such as push, pop, peek, display using a linked list.

AIM:

To implement a last-in, first-out stack using linked nodes and a menu for push, pop, peek, and display.

CODE:

```
class Node:
    def __init__(self, value, next_node=None):
        self.value, self.next = value, next_node


class LinkedStack:
    def __init__(self):
        self.top = None

    def push(self, value):
        self.top = Node(value, self.top)

    def pop(self):
        if self.top is None:
            return None
        value, self.top = self.top.value, self.top.next
        return value

    def peek(self):
        return self.top.value if self.top else None

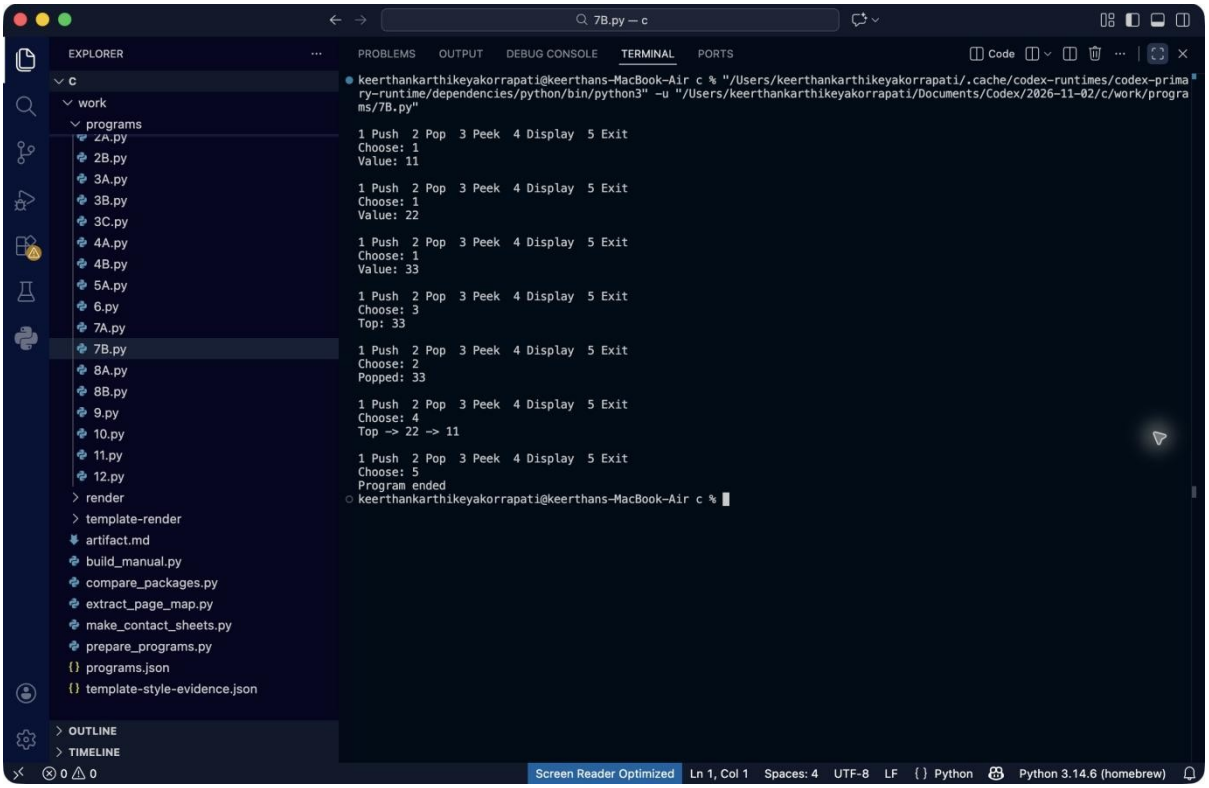
    def display(self):
        values, current = [], self.top
        while current:
            values.append(str(current.value))
            current = current.next
        print("Top ->", " -> ".join(values) if values else "Stack is empty")


def main():
    stack = LinkedStack()
    while True:
        print("\n1 Push  2 Pop  3 Peek  4 Display  5 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            stack.push(int(input("Value: ")))
        elif choice == "2":
            print("Popped:", stack.pop())
        elif choice == "3":
            print("Top:", stack.peek())
        elif choice == "4":
            stack.display()
        elif choice == "5":
            print("Program ended")
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 7(B)



WEEK – 8

PROGRAM-8(A):

Queue Using Array and Linked List

QUESTION:

8(A). Write a python program to implement Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

AIM:

To implement first-in, first-out queue operations with both an array-backed queue and a linked queue.

CODE:

```
class ArrayQueue:
    def __init__(self):
        self.items = []

    def enqueue(self, value):
        self.items.append(value)

    def dequeue(self):
        return self.items.pop(0) if self.items else None

    def peek(self):
        return self.items[0] if self.items else None

    def display(self):
        print("Front ->", self.items, "<- Rear")


class Node:
    def __init__(self, value):
        self.value, self.next = value, None


class LinkedQueue:
    def __init__(self):
        self.front = self.rear = None

    def enqueue(self, value):
        node = Node(value)
        if self.rear:
            self.rear.next = node
        else:
            self.front = node
        self.rear = node

    def dequeue(self):
        if self.front is None:
            return None
        value, self.front = self.front.value, self.front.next
        if self.front is None:
            self.rear = None
        return value

    def peek(self):
        return self.front.value if self.front else None

    def display(self):
        values, current = [], self.front
```

```
        while current:
            values.append(current.value)
            current = current.next
        print("Front ->", values, "<- Rear")

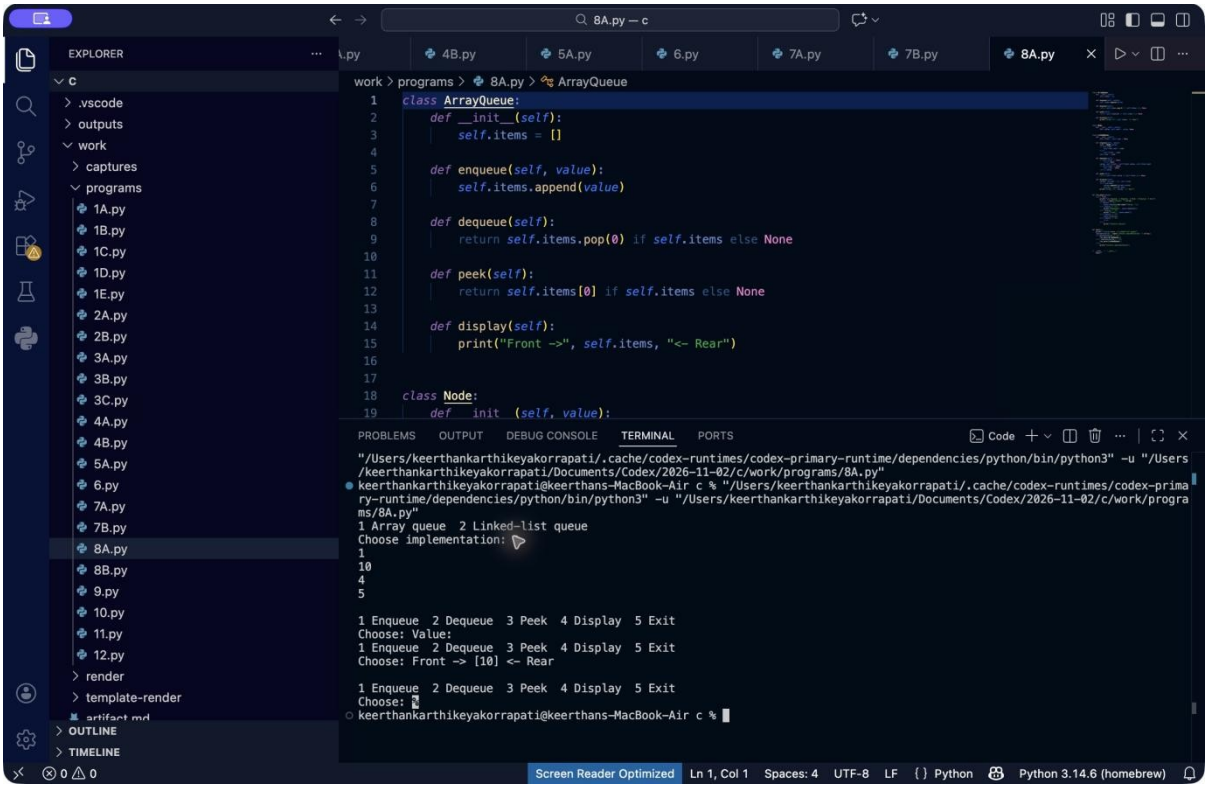

def run_menu(queue):
    while True:
        print("\n1 Enqueue  2 Dequeue  3 Peek  4 Display  5 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            queue.enqueue(int(input("Value: ")))
        elif choice == "2":
            print("Dequeued:", queue.dequeue())
        elif choice == "3":
            print("Front:", queue.peek())
        elif choice == "4":
            queue.display()
        elif choice == "5":
            break
        else:
            print("Invalid choice")


def main():
    print("1 Array queue  2 Linked-list queue")
    implementation = input("Choose implementation: ").strip()
    if implementation == "1":
        run_menu(ArrayQueue())
    elif implementation == "2":
        run_menu(LinkedQueue())
    else:
        print("Invalid implementation")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 8(A)



PROGRAM-8(B):

Circular Queue Using Array and Linked List

QUESTION:

8(B). Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.

AIM:

To implement circular queue operations using a fixed-size array and a circular linked list.

CODE:

```
class CircularArrayQueue:
    def __init__(self, capacity=5):
        self.data = [None] * capacity
        self.front = 0
        self.size = 0

    def enqueue(self, value):
        if self.size == len(self.data):
            return False
        rear = (self.front + self.size) % len(self.data)
        self.data[rear] = value
        self.size += 1
        return True

    def dequeue(self):
        if self.size == 0:
            return None
        value = self.data[self.front]
        self.data[self.front] = None
        self.front = (self.front + 1) % len(self.data)
        self.size -= 1
        return value

    def peek(self):
        return self.data[self.front] if self.size else None

    def display(self):
        values = [self.data[(self.front + i) % len(self.data)] for i in range(self.size)]
        print("Front ->", values, "<- Rear")


class Node:
    def __init__(self, value):
        self.value, self.next = value, None


class CircularLinkedQueue:
    def __init__(self):
        self.rear = None
        self.size = 0

    def enqueue(self, value):
        node = Node(value)
        if self.rear is None:
            node.next = node
            self.rear = node
        else:
            node.next = self.rear.next
            self.rear.next = node
```

```
            self.rear = node
        self.size += 1

    def dequeue(self):
        if self.rear is None:
            return None
        front = self.rear.next
        if front is self.rear:
            self.rear = None
        else:
            self.rear.next = front.next
        self.size -= 1
        return front.value

    def peek(self):
        return self.rear.next.value if self.rear else None

    def display(self):
        values, current = [], self.rear.next if self.rear else None
        for _ in range(self.size):
            values.append(current.value)
            current = current.next
        print("Front ->", values, "<- Rear")

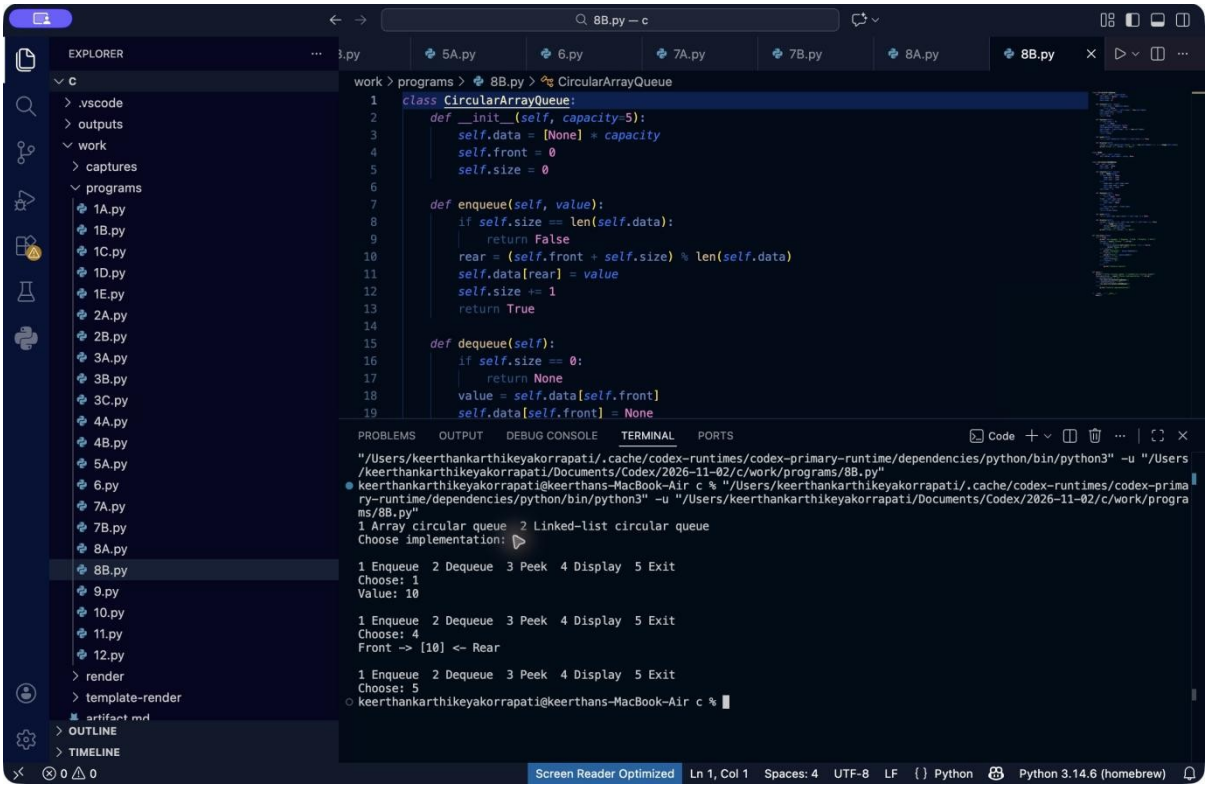

def run_menu(queue):
    while True:
        print("\n1 Enqueue  2 Dequeue  3 Peek  4 Display  5 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            if queue.enqueue(int(input("Value: "))) is False:
                print("Queue is full")
        elif choice == "2":
            print("Dequeued:", queue.dequeue())
        elif choice == "3":
            print("Front:", queue.peek())
        elif choice == "4":
            queue.display()
        elif choice == "5":
            break
        else:
            print("Invalid choice")


def main():
    print("1 Array circular queue  2 Linked-list circular queue")
    implementation = input("Choose implementation: ").strip()
    if implementation == "1":
        run_menu(CircularArrayQueue())
    elif implementation == "2":
        run_menu(CircularLinkedQueue())
    else:
        print("Invalid implementation")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 8(B)



WEEK – 9

PROGRAM-9:

Binary Search Tree Operations

QUESTION:

Write a Python program to implement a Binary Search Tree and perform insertion, deletion, in-order, pre-order and post-order traversals, count leaf and internal nodes, and find the tree height.

AIM:

To construct a binary search tree and demonstrate its update, traversal, counting, and height operations.

CODE:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None


class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        def add(node):
            if node is None:
                return Node(value)
            if value < node.value:
                node.left = add(node.left)
            elif value > node.value:
                node.right = add(node.right)
            return node
        self.root = add(self.root)

    def delete(self, value):
        def remove(node):
            if node is None:
                return None
            if value < node.value:
                node.left = remove(node.left)
            elif value > node.value:
                node.right = remove(node.right)
            else:
                if node.left is None:
                    return node.right
                if node.right is None:
                    return node.left
                successor = node.right
                while successor.left:
                    successor = successor.left
                node.value = successor.value
                node.right = delete_min(node.right)
            return node

        def delete_min(node):
            if node.left is None:
                return node.right
            node.left = delete_min(node.left)
            return node
```

```
        self.root = remove(self.root)

    def traversals(self):
        result = {"in-order": [], "pre-order": [], "post-order": []}
        def visit(node):
            if node is None:
                return
            result["pre-order"].append(node.value)
            visit(node.left)
            result["in-order"].append(node.value)
            visit(node.right)
            result["post-order"].append(node.value)
        visit(self.root)
        return result

    def leaf_count(self):
        def count(node):
            if node is None:
                return 0
            if node.left is None and node.right is None:
                return 1
            return count(node.left) + count(node.right)
        return count(self.root)

    def internal_count(self):
        def count(node):
            if node is None or (node.left is None and node.right is None):
                return 0
            return 1 + count(node.left) + count(node.right)
        return count(self.root)

    def height(self):
        def measure(node):
            if node is None:
                return 0
            return 1 + max(measure(node.left), measure(node.right))
        return measure(self.root)

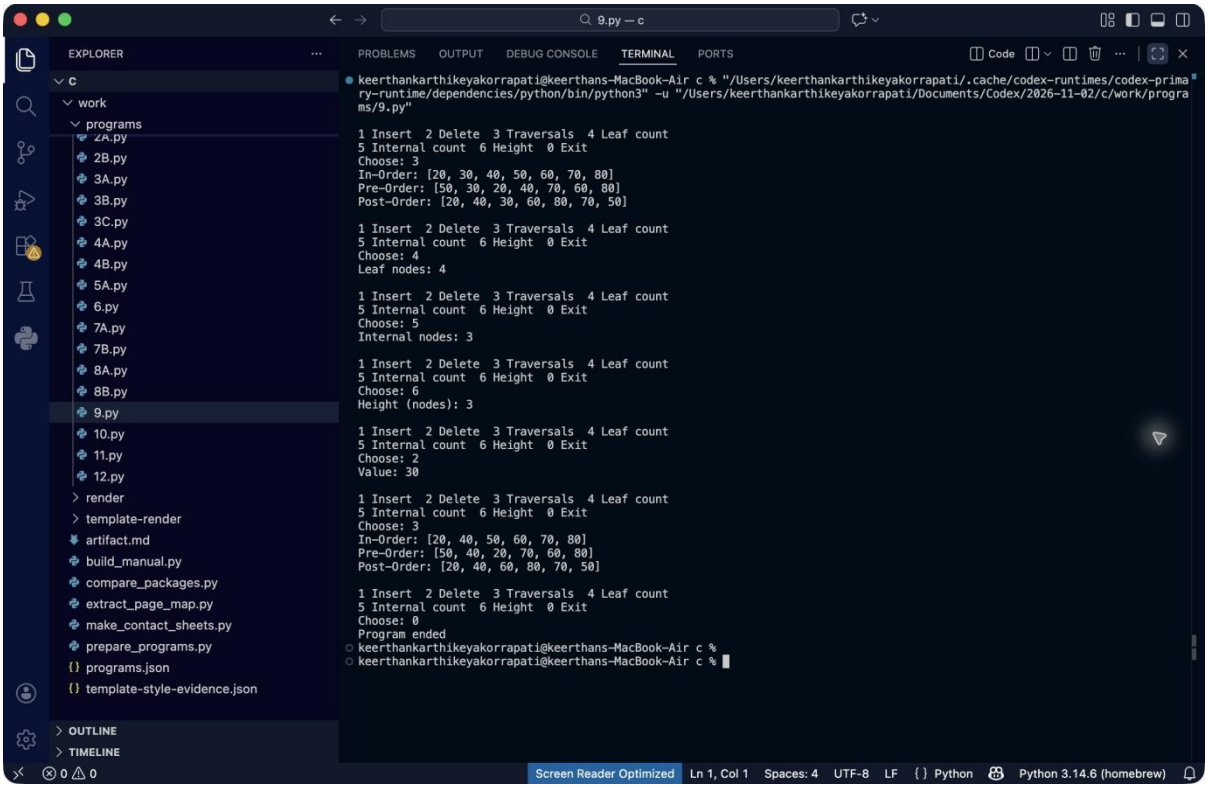

def main():
    tree = BinarySearchTree()
    for value in [50, 30, 70, 20, 40, 60, 80]:
        tree.insert(value)
    while True:
        print("\n1 Insert  2 Delete  3 Traversals  4 Leaf count")
        print("5 Internal count  6 Height  0 Exit")
        choice = input("Choose: ").strip()
        if choice == "1":
            tree.insert(int(input("Value: ")))
        elif choice == "2":
            tree.delete(int(input("Value: ")))
        elif choice == "3":
            for name, values in tree.traversals().items():
                print(f"{name.title()}: {values}")
        elif choice == "4":
            print("Leaf nodes:", tree.leaf_count())
        elif choice == "5":
            print("Internal nodes:", tree.internal_count())
        elif choice == "6":
            print("Height (nodes):", tree.height())
        elif choice == "0":
            print("Program ended")
            break
        else:
            print("Invalid choice")


if __name__ == "__main__":
```

```
main()
```

OUTPUT:

VS Code run for Program 9



WEEK – 10

PROGRAM-10:

AVL Tree Operations

QUESTION:

Write a menu-driven Python program to implement an AVL Tree and perform insertion, deletion, tree traversals, count leaf and internal nodes, and find the tree height.

AIM:

To maintain a height-balanced AVL tree through rotations and support insertion, deletion, traversal, counting, and height operations.

CODE:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = self.right = None
        self.height = 1


def height(node):
    return node.height if node else 0


def update(node):
    node.height = 1 + max(height(node.left), height(node.right))
    return node


def rotate_right(root):
    new_root = root.left
    root.left = new_root.right
    new_root.right = root
    update(root)
    return update(new_root)


def rotate_left(root):
    new_root = root.right
    root.right = new_root.left
    new_root.left = root
    update(root)
    return update(new_root)


def rebalance(node):
    update(node)
    balance = height(node.left) - height(node.right)
    if balance > 1:
        if height(node.left.left) < height(node.left.right):
            node.left = rotate_left(node.left)
        return rotate_right(node)
    if balance < -1:
        if height(node.right.right) < height(node.right.left):
            node.right = rotate_right(node.right)
        return rotate_left(node)
    return node
```

```
class AVLTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        def add(node):
            if node is None:
                return Node(value)
            if value < node.value:
                node.left = add(node.left)
            elif value > node.value:
                node.right = add(node.right)
            else:
                return node
            return rebalance(node)
        self.root = add(self.root)

    def delete(self, value):
        def remove(node):
            if node is None:
                return None
            if value < node.value:
                node.left = remove(node.left)
            elif value > node.value:
                node.right = remove(node.right)
            else:
                if node.left is None:
                    return node.right
                if node.right is None:
                    return node.left
                successor = node.right
                while successor.left:
                    successor = successor.left
                node.value = successor.value
                node.right = remove_value(node.right, successor.value)
            return rebalance(node)
        self.root = remove(self.root)

    def traversals(self):
        result = {"in-order": [], "pre-order": [], "post-order": []}
        def visit(node):
            if node is None:
                return
            result["pre-order"].append(node.value)
            visit(node.left)
            result["in-order"].append(node.value)
            visit(node.right)
            result["post-order"].append(node.value)
        visit(self.root)
        return result

    def count(self, leaves=False):
        def measure(node):
            if node is None:
                return 0
            is_leaf = node.left is None and node.right is None
            if leaves:
                return int(is_leaf) + measure(node.left) + measure(node.right)
            return int(not is_leaf) + measure(node.left) + measure(node.right)
        return measure(self.root)

    def tree_height(self):
        return height(self.root)


def remove_value(node, value):
    if value < node.value:
        node.left = remove_value(node.left, value)
    elif value > node.value:
```

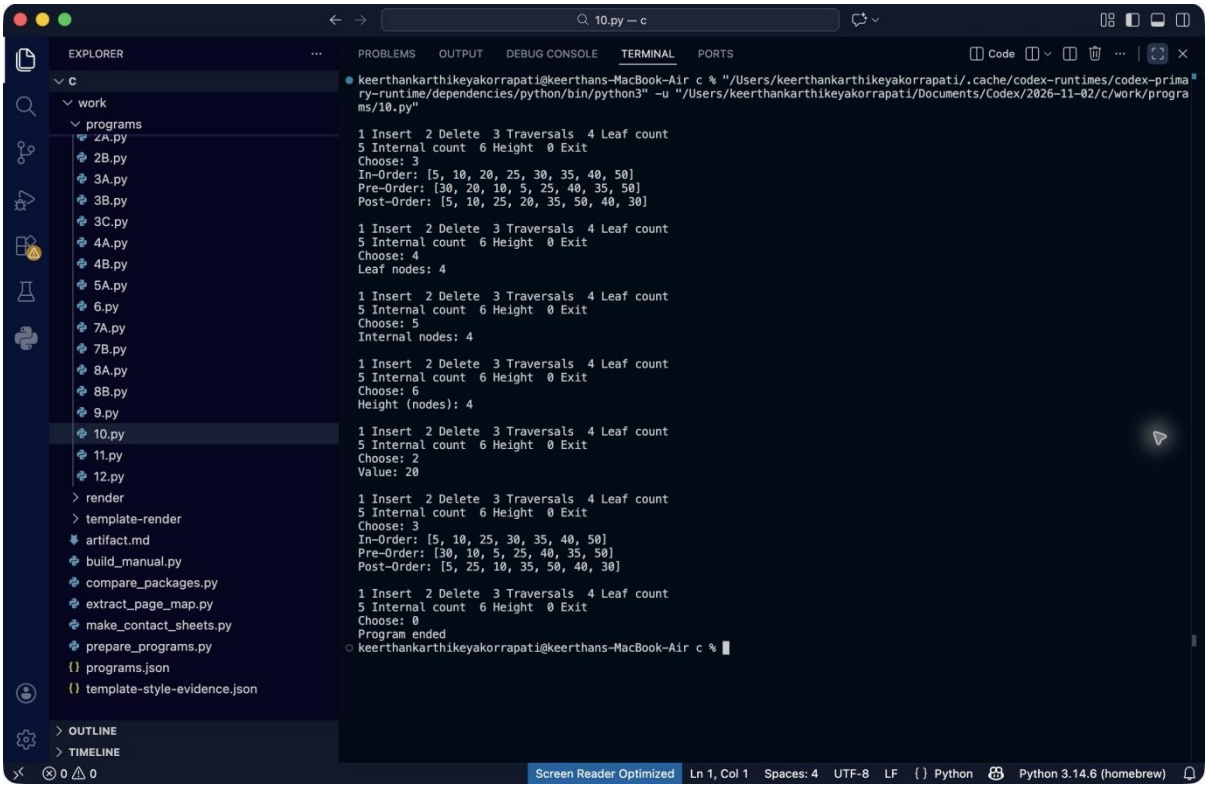
```
            node.right = remove_value(node.right, value)
        else:
            if node.left is None:
                return node.right
            if node.right is None:
                return node.left
            successor = node.right
            while successor.left:
                successor = successor.left
            node.value = successor.value
            node.right = remove_value(node.right, successor.value)
        return rebalance(node)


    def main():
        tree = AVLTree()
        for value in [30, 20, 40, 10, 25, 35, 50, 5]:
            tree.insert(value)
        while True:
            print("\n1 Insert  2 Delete  3 Traversals  4 Leaf count")
            print("5 Internal count  6 Height  0 Exit")
            choice = input("Choose: ").strip()
            if choice == "1":
                tree.insert(int(input("Value: ")))
            elif choice == "2":
                tree.delete(int(input("Value: ")))
            elif choice == "3":
                for name, values in tree.traversals().items():
                    print(f"{name.title()}: {values}")
            elif choice == "4":
                print("Leaf nodes:", tree.count(leaves=True))
            elif choice == "5":
                print("Internal nodes:", tree.count())
            elif choice == "6":
                print("Height (nodes):", tree.tree_height())
            elif choice == "0":
                print("Program ended")
                break
            else:
                print("Invalid choice")


    if __name__ == "__main__":
        main()
```

OUTPUT:

VS Code run for Program 10



WEEK – 11

PROGRAM-11:

Binary Max Heap and Min Heap

QUESTION:

Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) with insertion, deletion of the root element, display of the array representation, and exit.

AIM:

To implement both max-heap and min-heap ordering using array storage, sift-up insertion, and root deletion.

CODE:

```
class BinaryHeap:
    def __init__(self, kind):
        self.values = []
        self.kind = kind

    def higher_priority(self, first, second):
        return first > second if self.kind == "max" else first < second

    def insert(self, value):
        self.values.append(value)
        child = len(self.values) - 1
        while child > 0:
            parent = (child - 1) // 2
            if not self.higher_priority(self.values[child], self.values[parent]):
                break
            self.values[child], self.values[parent] = self.values[parent], self.values[child]
            child = parent

    def delete_root(self):
        if not self.values:
            return None
        root = self.values[0]
        last = self.values.pop()
        if self.values:
            self.values[0] = last
            parent = 0
            while True:
                left, right = 2 * parent + 1, 2 * parent + 2
                best = parent
                if left < len(self.values) and self.higher_priority(self.values[left], self.values[best]):
                    best = left
                if right < len(self.values) and self.higher_priority(self.values[right], self.values[best]):
                    best = right
                if best == parent:
                    break
                self.values[parent], self.values[best] = self.values[best], self.values[parent]
                parent = best
        return root

    def display(self):
        print(f"{self.kind.title()} heap array:", self.values)

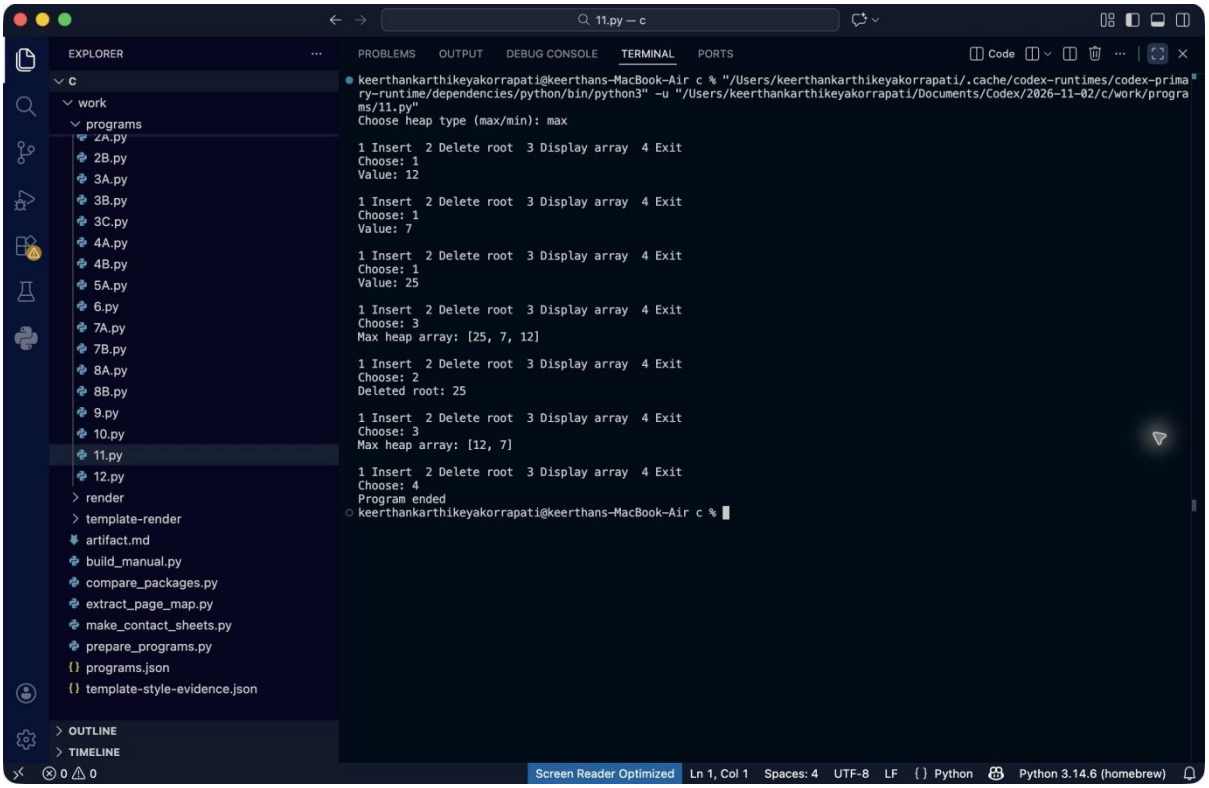

def main():
    kind = input("Choose heap type (max/min): ").strip().lower()
    if kind not in {"max", "min"}:
```

```
            print("Invalid heap type")
            return
        heap = BinaryHeap(kind)
        while True:
            print("\n1 Insert  2 Delete root  3 Display array  4 Exit")
            choice = input("Choose: ").strip()
            if choice == "1":
                heap.insert(int(input("Value: ")))
            elif choice == "2":
                print("Deleted root:", heap.delete_root())
            elif choice == "3":
                heap.display()
            elif choice == "4":
                print("Program ended")
                break
            else:
                print("Invalid choice")


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 11



WEEK – 12

PROGRAM-12:

Hash Table Collision Resolution

QUESTION:

Write a program to implement the three hash table collision resolution techniques: separate chaining, linear probing, and quadratic probing.

AIM:

To compare separate chaining, linear probing, and quadratic probing when keys collide in a hash table.

CODE:

```
class ChainingTable:
    def __init__(self, size):
        self.table = [[] for _ in range(size)]

    def insert(self, key):
        bucket = key % len(self.table)
        if key not in self.table[bucket]:
            self.table[bucket].append(key)

    def search(self, key):
        return key in self.table[key % len(self.table)]


class ProbingTable:
    def __init__(self, size, method):
        self.table = [None] * size
        self.method = method

    def insert(self, key):
        size = len(self.table)
        start = key % size
        for attempt in range(size):
            offset = attempt if self.method == "linear" else attempt * attempt
            index = (start + offset) % size
            if self.table[index] in (None, key):
                self.table[index] = key
                return True
        return False

    def search(self, key):
        size = len(self.table)
        start = key % size
        for attempt in range(size):
            offset = attempt if self.method == "linear" else attempt * attempt
            index = (start + offset) % size
            if self.table[index] is None:
                return False
            if self.table[index] == key:
                return True
        return False


def main():
    size = 7
    keys = [10, 17, 24, 31]
    print("Table size:", size)
```

```
    print("Keys with the same initial hash:", keys)

    chain = ChainingTable(size)
    for key in keys:
        chain.insert(key)
    print("\nSeparate chaining:")
    for index, bucket in enumerate(chain.table):
        print(f"{index}: {bucket}")
    print("Search 24:", chain.search(24))

    for method in ("linear", "quadratic"):
        table = ProbingTable(size, method)
        for key in keys:
            table.insert(key)
        print(f"\n{method.title()} probing:")
        print(table.table)
        print("Search 24:", table.search(24))


if __name__ == "__main__":
    main()
```

OUTPUT:

VS Code run for Program 12

